Math Practice Web Application

Development Report | Individual AI-Assisted Project | October 2026

1. Development approach

I developed a browser-based Math Practice application for children ages 7–12 using ChatGPT for coding assistance and VS Code with a PowerShell terminal to run and verify the application. I provided the assignment instructions, software specification, implementation checklist, and an example AI development transcript. ChatGPT produced project files in incremental ZIP packages, which I integrated into my local project and tested. I used Git commits to preserve development checkpoints.

The original implementation checklist contained 22 phases. After completing the first five, I requested that the remaining work be combined into larger stages without reducing the requirements. We finished with a 10-phase workflow covering the React/TypeScript/Vite foundation, question generation, interactive cards, sessions, persistence, statistics, mistake review, settings, accessibility, and deployment. I maintained the original checklist and development log so progress could still be traced to the specification.

2. Prompting strategy

Scoped implementation: In my Phase 4 prompt, I explicitly requested all four arithmetic operations, level-specific ranges, nonnegative subtraction, whole-number division, reusable utilities, and boundary tests. I also stated that multiple-choice cards should not be added yet. This helped keep the math engine separate from later interface work.

Integration with constraints: My revised Phase 6 prompt combined interactive cards, practice-session management, and summaries. It specifically required preserving earlier functionality, preventing duplicate scoring, supporting continuous practice, and deferring persistent history until the next stage. This clarified both the feature scope and what should remain unchanged.

Quality-focused refinement: The Phase 9 prompt requested a comprehensive requirements audit, regression tests, accessibility and mobile improvements, refresh recovery, and accurate updates to the TODO and development log. I repeatedly emphasized testing and avoiding unnecessary feature changes rather than relying on code generation alone.

3. Debugging and iteration

The process produced real errors that I had to investigate. In Phase 3, missing `@testing-library/user-event` caused test and TypeScript build failures; installing that dependency solved the problem. In Phase 6, a test detected multiple mounted QuestionCard components, so we added test cleanup. ESLint also identified an unused result-callback parameter, which I removed.

Phase 8 exposed a malformed difficulty-saving expression, two outdated test expectations, and validation alerts that affected test queries. Phase 9 initially passed 85 of 87 tests; the remaining failures occurred because a selected answer had a new accessible label and a direct navigation test now encountered the session-recovery feature. We updated those tests to reflect the intended behavior, rather than deleting them. Sharing exact terminal output made the corrections more targeted.

4. Testing and verification

I used Vitest and React Testing Library to test arithmetic generators, answer choices, card interaction, sessions, storage, statistics, mistakes, mastery, settings, and integrated user flows. For the final project package, my local terminal reported **87 tests passing across 17 test files**, a successful ESLint run, and a successful TypeScript/Vite production build. A separate formatting check initially reported 47 files needing Prettier formatting, and I was advised to format and repeat verification. These local results were provided to ChatGPT; they were not reproduced in its environment.

I also tested the application manually: completed practice sessions, checked score summaries and saved statistics, reset history, reviewed previously missed questions, and checked mistake mastery. I confirmed the apparently surprising behavior that a wrong multiple-choice selection can be marked correct by clicking “I Got It Right.” We retained that behavior after checking the specification, because scoring is intentionally self-reported. In another test, a five-question mistake-review session ended after three correct similar questions once the only mistake was mastered. The default three-success mastery rule is in the specification, but early session termination is an implementation choice; I documented it without changing the app.

The project was pushed to my existing class GitHub repository and deployed to Netlify. I reported that the deployed application worked. This report does not claim an independent external site audit or testing on every browser or assistive technology.

5. Reflection and limitations

AI assistance made it much faster to build features, write tests, and diagnose errors, especially when requests specified exact requirements and boundaries. Its main limitation was reliability: generated packages occasionally omitted dependencies, included malformed expressions, or left tests inconsistent with new features. Running the application myself and comparing behavior against the specification were essential. Keeping development incremental and retaining actual debugging evidence made errors easier to trace.

The application stores data in the browser, so history is local to the browser and origin rather than shared across accounts. Active-session recovery restores the question and counters, but not necessarily a selected option or flipped card face after refresh. The final dependency audit also reported three advisories in the Vitest testing toolchain (one moderate and two critical); I did not apply the suggested forced major-version upgrade without regression testing. These limitations are documented for transparency.